

Research Progress Report

Project:	IsoShard Proxy: A Distributed CP-Mode Transaction Coordinator & SQL Optimizer on AWS
Proponent:	Sarthak Mushrif
Reporting Period:	Weeks 16 & 17
Hours Logged:	20 Hours

1. Executive Summary of Progress

During the eighth reporting period, development shifted toward read-path optimizations and complex query aggregation. The proxy's capabilities were significantly expanded to support cross-shard JOIN operations, a historically challenging feature in distributed databases, achieved via an in-memory Hash Join implementation. Concurrently, a localized caching tier was introduced to intercept high-frequency read queries, drastically reducing network I/O to the AWS RDS instances. Finally, aggressive TCP load testing was conducted against the AWS Network Load Balancer (NLB) infrastructure to validate the asynchronous Netty architecture under high concurrency.

2. Detailed Task Breakdown (20 Hours)

- **Cross-Shard JOIN Implementation (8 Hours):** Upgraded the AST SqlVisitor to detect when a SELECT statement joins tables residing on different physical shards. Developed the IsoShardJoinAggregator, which uses CompletableFuture to query both target shards simultaneously. The proxy then performs an in-memory Hash Join on the resulting row sets before serializing the final combined payload back to the client.
- **High-Concurrency Load Testing (6 Hours):** Authored a custom multi-threaded Netty client to bypass HTTP overhead and blast the AWS NLB with raw binary TCP packets. Measured throughput up to 12,000 Transactions Per Second (TPS). Tuned the underlying EC2 HikariCP pool sizing per instance to prevent database connection starvation under maximum load.
- **In-Memory Query Caching (6 Hours):** Integrated Caffeine Cache (a high-performance Java caching library) directly into the proxy layer. Designed a "read-through" architecture where exact-match SELECT queries bypass the consistent hashing router entirely if a valid result set exists in the JVM cache.

3. Challenges & Roadblocks

- **JVM Memory Exhaustion on Large JOINS:** Initial testing of the cross-shard JOIN logic resulted in OutOfMemoryError exceptions when joining tables with over 500,000 rows, as the proxy attempted to load both full datasets into RAM. This was mitigated by implementing a chunked, paginated streaming cursor in the JDBC fetch logic, allowing the proxy to merge and flush data to the client in smaller byte batches.
- **Cache Invalidation Races:** The introduction of the cache created "dirty read" scenarios where an UPDATE transaction modified a row, but the cache continued serving the stale SELECT result. Engineered a strict cache-invalidation hook within Phase 1 of the 2PC protocol to purge affected table keys before the write commit finalizes.

4. Objectives for Next Reporting Period (Weeks 18 & 19)

- Implement Distributed Deadlock Detection logic to identify and terminate circular lock dependencies between competing distributed transactions.
- Expand the ANTLR4 compiler frontend to parse and optimize GROUP BY and ORDER BY aggregation clauses.
- Instrument the proxy application with Micrometer to export real-time metrics (latency, cache hit rate, TPS) to AWS CloudWatch for live dashboarding.

Appendix A: Technical Artifact - Cross-Shard Hash Join Aggregator

The following Java snippet demonstrates the asynchronous fetch and in-memory merge logic used to perform a distributed JOIN without pulling the data back to the client application layer.

```
1 import java.util.*;
2 import java.util.concurrent.CompletableFuture;
3 import java.util.stream.Collectors;
4
5 public class IsoShardJoinAggregator {
6
7     /**
8      * Executes queries on two different shards asynchronously and performs
9      * an in-memory Hash Join on the specified join column.
10    */
11    public CompletableFuture<List<Map<String, Object>>> executeDistributedJoin(
12        String shardA, String queryA, String joinColA,
13        String shardB, String queryB, String joinColB,
14        ShardManager manager) {
15
16        // Fire both network requests in parallel
17        CompletableFuture<List<Map<String, Object>>> futureA = manager.
18            asyncQuery(shardA, queryA);
19        CompletableFuture<List<Map<String, Object>>> futureB = manager.
20            asyncQuery(shardB, queryB);
21
22        return futureA.thenCombine(futureB, (resultSetA, resultSetB) -> {
23            // Build the Hash Map from the smaller result set (Set B for this
24            // example)
25            Map<Object, Map<String, Object>> hashIndex = resultSetB.stream()
26                .collect(Collectors.toMap(row -> row.get(joinColB), row -> row));
27
28            List<Map<String, Object>> joinedResults = new ArrayList<>();
29
30            // Probe the Hash Map using Set A
31            for (Map<String, Object> rowA : resultSetA) {
32                Object joinValue = rowA.get(joinColA);
33                if (hashIndex.containsKey(joinValue)) {
34                    Map<String, Object> combinedRow = new HashMap<>(rowA);
35                    combinedRow.putAll(hashIndex.get(joinValue));
36                    joinedResults.add(combinedRow);
37                }
38            }
39
40            return joinedResults;
41        });
42    }
43 }
```

Listing 1: IsoShardJoinAggregator.java